

Sistemas Embebidos

Trabajo especial :
Reproductor WAV con Ecualización

Integrantes

Marchesotti, Guido Daniel



FACULTAD
DE INGENIERÍA



UNIVERSIDAD
NACIONAL
DE LA PLATA

1 Explicación

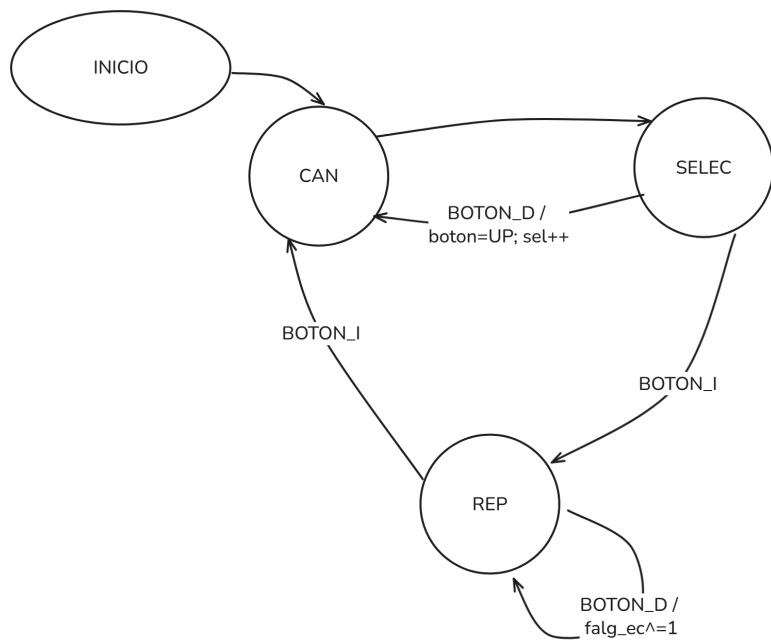


Figure 1: Maquina de estados

Este trabajo se implementó utilizando tres tareas y una interrupción. Una de las tareas se encarga de leer los datos provenientes de la interfaz de usuario, como el botón y el potenciómetro. La segunda tarea, utilizando la librería SSD1306, controla la pantalla OLED. Por último, la tercera tarea gestiona el lector de tarjeta SD, abriendo el archivo correspondiente según la selección realizada.

Las tareas fueron distribuidas de esta forma con el objetivo de evitar el uso compartido de recursos entre ellas, minimizando así la posibilidad de errores por accesos concurrentes.

Por su parte, la interrupción se utiliza para actualizar los valores del registro comparador del PWM, garantizando la correcta generación de la señal.

En la Figura 1 se muestra una máquina de estados que describe el funcionamiento general del proyecto

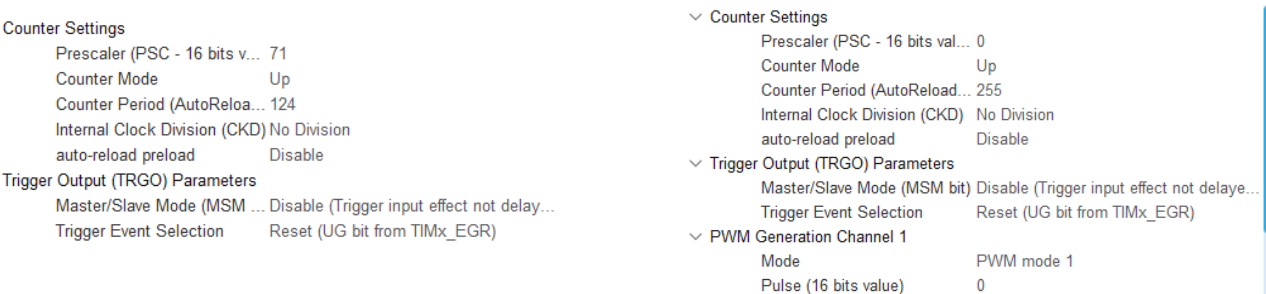


Figure 2: Configuración Timer2

Figure 3: Configuración Timer3

En las figuras 2 y 3 se muestra la configuración de los timers utilizados en el proyecto. El Timer 3 se configuró en modo PWM, sin prescaler, operando a la frecuencia del microcontrolador (72MHz), y con un valor máximo de 255 en el contador.

Por su parte, el Timer 2 se configuró para generar una interrupción cada $125\mu s$, la cual se utiliza para actualizar el valor del registro comparador CCR1 del Timer 3, modificando así el ciclo de trabajo del PWM.

```

38 #define BUFFERSIZE 128
39
40 #define BARRA_ANCHO 4
41 #define BARRA_MAX_ALTURA 64
42 #define CAIDA_PICO 1
43
44 #define SIZE_FFT 64
45 #define FFT_COL 32
46
47 #define velocidad_fall 4
48
49 typedef enum {
50     NING,
51     UP,
52     DnN
53 }botones_e;
54
55 typedef enum{
56     INICIO,
57     CAN,
58     REP,
59     SELEC
60 }mensajes_l;
61
62 mensajes_l men=INICIO;
63 botones_e boton=NING;
64
65 char archivos[5][11];
66
67 uint16_t idx;
68
69 uint8_t bufferDn[BUFFERSIZE],bufferUp[BUFFERSIZE];
70
71
72 uint8_t *puntero_escritura = bufferUp;
73 uint8_t *puntero_lectura = bufferDn;

```

Figure 4: parte 1. Variables globales

```

75 cmpx V[SIZE_FFT]; //buffer para hacer la fft
76
77 uint8_t barras[FFT_COL], barras_peak[FFT_COL],y[FFT_COL];
78 uint8_t x[FFT_COL]={4,8,12,16,20,24,28,32,36,40,44,48,52,46,60,64,
79     68,72,76,80,84,88,92,96,100,104,108,112,116,120,124,128};
80
81 uint8_t flag_ec,ready, clse,sel; // flags generales
82
83 uint32_t vadc;
84
85 float ganancia_low = 1.4;
86 float ganancia_high = 1.4;
87

```

Figure 5: parte 2. Variables globales

En las Figuras 4 y 5 se muestran las variables globales que se usaran en el código en diferentes tareas e interrupción.

Las variables *men* y *boton*, respectivamente, representan el estado actual del sistema y el botón presionado por el usuario. La variable *archivos* contiene la lista de canciones que tendrá en la memoria. Se definieron también dos buffers para trabajar en modo ping-pong, junto con dos punteros que permiten conmutar entre ellos de manera eficiente durante la reproducción.

A su vez, se declararon varias variables tipo "flags" que cumplen funciones específicas: elegir la banda de frecuencia de la ecualización, indicar cuando el buffer este listo para reproducirse, imprimir la barras de la fft, señalar cuando se tenga que cerrar el archivo de música y para la selección de la canción.

```

647 void leerboton(void *argument)
648 {
649     /* USER CODE BEGIN leerboton */
650     vTaskSetApplicationTaskTag( NULL, (void*) TAG_Boton);
651     /* Infinite loop */
652
653     uint8_t lastState1 = 1, lastState2 = 1;
654     for(;;)
655     {
656         UBaseType_t Stack_res_boton=uxTaskGetStackHighWaterMark(NULL);
657
658         //lee el ADC
659         if(men==REP){
660             vadc=ADC1->DR;
661
662             if(flag_ec){
663                 ganancia_high= 0.2 + (float)(vadc/ 4049.0f)*(1.4f-0.2) ;
664             }else{
665                 ganancia_low= 0.2 + (float)(vadc/ 4049.0f)*(1.4f-0.2);
666             }
667         }
668
669
670         uint8_t currentState1 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_12);
671         uint8_t currentState2 = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_11);
672
673         // Botón 1 liberado
674         if (lastState1 == 0 && currentState1 == 1) {
675             boton= UP;
676         }
677
678         // Botón 2 liberado
679         if (lastState2 == 0 && currentState2 == 1) {
680             boton= DOWN;
681         }
682
683         lastState1 = currentState1;
684         lastState2 = currentState2;
685
686         osDelay(95);
687     }

```

Figure 6: Tarea interfaz

En la Figura6, se presenta la tarea que se encarga de la interfaz, la cual lee el valor del ADC en cada ejecución de la tarea y detecta los flancos de subida de los botones, almacenándolos en la variable *boton*. Después dentro de la tarea de la pantalla se procesarán los eventos de suelta del botón.

```

705 void pantalla(void *argument)
706 {
707     /* USER CODE BEGIN pantalla */
708
709     vTaskSetApplicationTaskTag( NULL, (void*) TAG_Pantalla);
710     uint8_t ver=10,k;
711
712     /* Infinite loop */
713     for(;;)
714     {
715         UBaseType_t Stack_res_Pan=uxTaskGetStackHighWaterMark(NULL);
716         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_SET);
717         switch (men) {
718
719             case CAN:
720                 SSD1306_Fill(SSD1306_COLOR_BLACK);
721                 for(uint8_t i=0;i<5;i++){
722                     SSD1306_GotoXY(5, 0+(ver*i));
723                     SSD1306_Puts(archivos[i], &Font_7x10, 1);
724                 }
725
726                 SSD1306_DrawFilledRectangle(107, (sel*10)+3, 4, 4, SSD1306_COLOR_WHITE);
727                 SSD1306_UpdateScreen();
728                 men=SELEC;
729                 break;
730
731             case REP:
732                 //graficar las barras
733                 FFT(&y, SIZE_FFT);
734                 calcular_barras_fft();
735                 SSD1306_Fill(SSD1306_COLOR_BLACK);
736                 for(k = 0; k < FFT_COL; k++) {
737                     if (barras_peak[k] <= barras[k]) {
738                         SSD1306_DrawFilledRectangle(x[k], 64 - barras_peak[k], BARRA_ANCHO - 1, barras_peak[k], SSD1306_COLOR_WHITE);
739                     }else{
740                         SSD1306_DrawFilledRectangle(x[k], y[k], BARRA_ANCHO - 1, barras[k], SSD1306_COLOR_WHITE);
741                     }
742                 }
743                 //marca para seleccion de banda
744                 if(flag_ec){
745                     SSD1306_DrawFilledRectangle(96,0,32,2, SSD1306_COLOR_WHITE);
746                 }else{
747                     SSD1306_DrawFilledRectangle(0,0,32,2, SSD1306_COLOR_WHITE);
748                 }
749             }
750         }
751     }

```

Figure 7: Parte 1. Tarea pantalla

La Figura 7 muestra la tarea que realizara la visualización en pantalla de los datos, que estará dividida dependiendo el estado en el que se encuentre. En un primer momento, la tarea imprime en pantalla una lista de hasta cinco archivos de audio almacenados en la memoria, en formato *.RAW*, los cuales fueron previamente cargados en la tarea de SD. Luego se pasa al estado de **SELEC** (Figura 8) donde el usuario puede utilizar los botones para navegar por la lista o seleccionar un archivo(botón izquierdo y derecho). Una vez realizada la selección, se cambia al estado de reproducción (**REP**), donde se calculara la FFT cada vez que se ejecute la tarea, con los valores cargados en la tarea SD, a su vez, se calcularan los valores de altura de las barras con la función *calcular_barras()* (Figura 9), donde se calculara el modulo de cada valor de la FFT, que sera el valor de la altura de la barra que se imprimirá. Después de calcular los valores se imprimen en pantalla las barras.

```

750     SSD1306_UpdateScreen();
751
752     if(boton==UP){
753         //hacer que se frene con este boton
754         men=CAN;
755         clse=1;
756         boton=NING;
757         osSemaphoreRelease(Sem_escHandle); //libero el semaforo por si esta bloqueando la tarea
758     }else if(boton==DWN){
759         //elegir la banda de frecuencias
760         flag_ec=1;
761         boton=NING;
762     }
763     break;
764
765     case SELEC:
766         switch (boton) {
767             case UP:
768                 //selecciona cancion
769                 men=REP;
770                 boton=NING;
771                 osSemaphoreRelease(Sem_escHandle);
772                 break;
773
774             case DWN:;
775                 //baja en las canciones
776                 sel++;
777                 if(sel>4){
778                     sel=0;
779                 }
780                 men=CAN;
781                 boton=NING;
782                 break;
783
784             default:
785                 break;
786         }
787         break;
788
789     default:
790         break;
791 }
792 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_RESET);
793 osDelay(150);
794 }

```

Figure 8: Parte 2 de la Tarea de la pantalla

```

592 void calcular_barras_fft(void) {
593     uint8_t modulo[FFT_COL];
594
595     for(uint8_t k = 1; k < FFT_COL+1; k++) {
596         modulo[k-1]= sqrtf(Y[k].real * Y[k].real + Y[k].imag * Y[k].imag);
597     }
598
599     for(uint8_t i=0;i<FFT_COL;i++){
600
601         barras[i] = log_table[modulo[i]]; // genera valor entre 0 y 63 de forma logaritmica
602         y[i] = BARRA_MAX_ALTURA-barras[i]; // valor de altura de la barra a graficar
603
604         if (barras[i] >= barras_peak[i]) {
605             barras_peak[i] = barras[i]; // nuevo pico
606         } else {
607             // "fall down" lento: baja 1 unidad cada vez que graficas
608             // if (barras_peak[i] > 0) barras_peak[i]--;
609             barras_peak[i] -= (barras_peak[i] > 0) ? velocidad_fall : 0;
610         }
611     }
612 }
613
614 }
615
616 }

```

Figure 9: Función para calcular las barras de la FFT

```

805 void lectura_sd(void *argument)
806 {
807     /* USER CODE BEGIN lectura_sd */
808     vTaskSetApplicationTaskTag( NULL, (void*) TAG_SD);
809
810     //variables for FatFs
811
812     FATFS FatFs; //Fatfs handle
813     FIL fil; //File handle
814     FRESULT fres; //Result after operations
815
816     DIR dir;
817     FILINFO fno;
818
819     UINT br; //numero de bytes leidos
820     uint8_t primera=0,valor;
821     uint16_t b;
822
823     int16_t salida_low,salida_high, entrada;
824
825
826     fres = f_mount(&FatFs, "", 1); //1=mount now
827     if (fres != FR_OK) {
828         while(1);
829     }
830
831     fres = f_opendir(&dir, "/"); // Abrir directorio raíz
832
833     if (fres == FR_OK) {
834         while (1) {
835             fres = f_readdir(&dir, &fno); // Leer siguiente archivo
836             if (fres != FR_OK || fno.fname[0] == 0) break; // Fin del directorio
837
838             if (strstr(fno.fname, ".RAW") != NULL) {
839                 strcpy(archivos[idx++], fno.fname);
840             }
841         }
842         f_closedir(&dir);
843         idx=0;
844         men=CAN;
845     }else{
846         while(1);
847     }
848 }
849

```

Figure 10: Parte 1.Tarea SD

Para la tarea de la SD (Figura 10), se utilizó la librería FATfs para la lectura de la tarjeta de memoria. Inicialmente se definieron variables propias de la librería, necesarias para poder utilizar las funciones de apertura y lectura de archivos.

En la primera ejecución de la tarea, se abre el directorio raíz donde se encuentran almacenados los archivos de audio. Mediante un bucle for, se recorren y se guardan los nombres en el arreglo *archivos*. Una vez finalizada esta operación, el directorio se cierra para liberar recursos del sistema, y se cambia el estado a *CAN*, lo que indica que los archivos ya pueden ser mostrados en la pantalla. Este bucle de carga de archivos se ejecuta una única vez, en la primera ejecución de la tarea.

```

852 for(;;)
853 {
854     UBaseType_t Stack_res_SD=uxTaskGetStackHighWaterMark(NULL); //valor en bytes
855     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_SET);
856
857
858     //se cierra el archivo cuando se apreta el boton stop o cuando termina la cancion
859     if(!clse || br==0){
860         f_close(&fil);
861         clse=0;
862         ready=0;
863         primera=0;
864         br=1;
865         men=CAN;
866     }
867
868     if(men==REP){
869
870         if(primer<1){
871             fres = f_open(&fil, archivos[sel], FA_READ); //abrimos archivo dependiendo el boton que se aprete
872             //verificamos que funcione
873             if (fres != FR_OK) {
874                 while(1);
875             }
876             primera++;
877         }
878
879         fres = f_read(&fil, puntero_escritura, BUFFERSIZE, &br);
880
881         if (fres != FR_OK) {
882             while(1);
883         }
884
885         for(b = 0; b<BUFFERSIZE; b++){
886             entrada=(int16_t )puntero_escritura[b]- 127;
887
888             //aplico los filtros de equalizacion
889             salida_low=(int16_t)(BiQuad(entrada, &filtro_LP)* ganancia_low);
890             salida_high=(int16_t)(BiQuad(entrada, &filtro_HP)* ganancia_high);
891
892             // Yolver a uints
893             valor=(uint8_t)(salida_low + salida_high) + 127;
894
895             if (valor < 0) valor = 0;
896             if (valor > 255) valor = 255;

```

Figure 11: Parte 2. Tarea SD

Una vez seleccionada la canción, se procede a abrir el archivo correspondiente para su reproducción. La variable *primera* se utiliza como indicador para asegurar que esta operación se realice únicamente una vez, al iniciar la reproducción. A continuación, se leen los primeros datos del archivo y se guardan en el buffer al que apunta *puntero_escritura*, sobre esta señal se le aplica dos filtros biquad, mediante los coeficientes previamente calculados para filtros centrados en 500Hz (pasa bajos) y 1500Hz (pasa altos), respectivamente. La salida de los filtros se multiplica a una ganancia ajustado mediante el potenciómetro, y estos mismos valores se cargan y se utilizan posteriormente para el cálculo de la FFT, que será visualizada en pantalla. Figura 11.


```

898         puntero_escritura[b]=valor;
899
900         if(b<SIZE_FFT){
901             Y[b].real=(float)valor;
902             Y[b].imag=0;
903         }
904     }
905
906     ready=1;
907
908     if(primera<2){
909         uint8_t *temp = puntero_lectura;
910         puntero_lectura = puntero_escritura;
911         puntero_escritura = temp;
912         primera++;
913     }
914 }
915
916
917 HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);
918
919 osSemaphoreAcquire(Sem_escHandle, oswaitForever);
920
921 }
922 /* USER CODE END lectura_sd */
923 }

```

Figure 12: Parte 3. Tarea SD

Al abrir el archivo por primera vez, y cargar el primer buffer, se realiza un intercambio de punteros entre los buffers para poder comenzar la carga del segundo buffer (ver Figura 12). A partir de ese momento, la carga de datos en los buffers se maneja utilizando un semáforo binario, que garantiza la protección y sincronización del acceso concurrente. Este semáforo bloquea la tarea SD, y será ejecutada cada vez que se libere el mismo, esto sucede únicamente cuando se produce el intercambio de punteros, lo cual ocurre, salvo en la primera lectura, solo después de que todos los datos del buffer actual hayan sido leídos por completo.

```

933 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
934 {
935     /* USER CODE BEGIN Callback 0 */
936     if (htim->Instance == TIM2) {
937
938         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
939
940         if(men==REP){
941             if(ready){
942
943                 TIM3->CCR1 = puntero_lectura[idx++];
944
945                 if (idx >= BUFFERSIZE) {
946                     idx = 0;
947                     uint8_t *temp = puntero_lectura;
948                     puntero_lectura = puntero_escritura;
949                     puntero_escritura = temp;
950                     osSemaphoreRelease(Sem_escHandle); // suelto el semáforo, cuando hay que cargar un nuevo buffer
951                     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
952                 }
953             }
954
955             }else {
956                 TIM3->CCR1 = 127; // Protección si no llegó el buffer a tiempo
957             }
958         }
959     /* USER CODE END Callback 0 */
960     if (htim->Instance == TIM4) {
961         HAL_IncTick();
962     }
963     /* USER CODE BEGIN Callback 1 */
964     /* USER CODE END Callback 1 */
965 }

```

Figure 13: Interrupción del timer

En la Figura 13 se observa la interrupción de los timers, el que se utiliza para el proyecto es el Timer2. Su función principal es actualizar el valor del registro CCR1 del Timer3 a una

frecuencia de 8kHz, utilizando los datos apuntados por la variable *puntero_lectura*. Una vez se leyó todo el buffer activo, se intercambian los punteros para que siga leyendo el otro buffer, al mismo tiempo se libera el semáforo para que la tarea SD pueda cargar nuevos datos en el buffer vacío.

2 Memoria y analizador lógico

Para conocer el uso de stack de cada tarea, se utilizó la función *uxTaskGetStackHighWaterMark*, la cual permite estimar cuánta memoria resta disponible en el stack de una tarea, midiendo la diferencia entre el punto de mayor uso y el final del stack asignado. Para la tarea de botones se le asignó un stack de 296 bytes, quedando 30 bytes libres, a la tarea de la pantalla 960 bytes, con 108 bytes libres, y para la tarea de la SD 3380 bytes, con 161bytes restantes. En cuanto al heap se le asignó un tamaño de 6552 bytes, de los cuales quedaron restando 1092 bytes.

En la Figura 14 se puede observar el espacio en memoria ocupado en FLASH y en RAM, viendo que no es posible agregar muchas funcionalidades mas, a menos que se optimicen algunas funciones, o se considere reducir el uso de la HAL, ya que esta genera funciones más extensas y menos eficientes en comparación con una implementación directa usando registros.

Region	Start address	End address	Size	Free	Used	Usage (%)
 RAM	0x20000000	0x20004fff	20 KB	4,12 KB	15,88 KB	79,41%
 FLASH	0x08000000	0x0800ffff	64 KB	1,7 KB	62,3 KB	97,35%

Figure 14: Uso de la memoria

En cuanto al funcionamiento del programa, se observa el tiempo de ejecución de cada tarea (Boton, Pantalla, SD e IDLE), como también los tiempos que tarda en procesar toda las tares (Time-sd y Time-pan), antes de iniciar la reproducción de una canción(Figura 15). Por otro lado, la Figura 16 muestra la misma información pero una vez seleccionada una canción y comenzada su reproducción.Lo que se ve en las Figuras de la interrupción no es cada ejecución puntual, sino el tiempo que tarda, el PWM, desde que empieza a leer hasta que termina de consumir todos lo valores del buffer. Cuando se calculo el factor de utilización de las tareas, el del botón resulto $\frac{64.458\mu s}{89.45ms} = 0.687m$, el de la tarea de la pantalla $\frac{58.65ms}{208.64ms} = 0.28$ y la tarea de la SD $\frac{4ms}{16ms} = 0.25$, mientras que la interrupción se realiza cada $125\mu s$. El factor de utilización total resulta de la suma de los factores individuales $(0.687m + 0.25 + 0.28) * 100 = 53\%$, y como se puede observar en la Figura 16 todavía hay tiempo restante en el sistema par realizar mas procesamiento. Para esta implementación se configuraron las tareas con distintas prioridades, utilizando que para las tareas mas importantes para el proyecto y para la que mas procesamiento deben hacer tengan mayor prioridad ,la tarea que se encarga de la interfaz (tarea_boton) se le asigna una prioridad *Normal*, mientras que la tarea de la pantalla se le dara una prioridad de *Normal1* y la SD prioridad *Normal2*, ya que esta ultima teniendo el mayor periodo y que si se retrasara se perderían valores para reproducir y se escucharía con mala calidad, necesitando que cada vez que pueda se ejecute y realice su funcion.

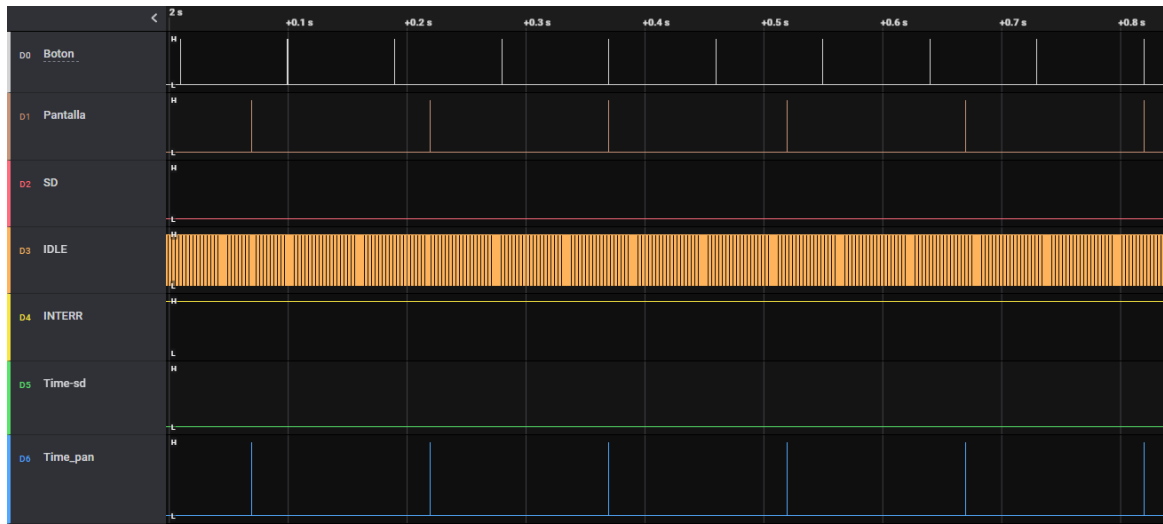


Figure 15: Logic Analyzer: sin música

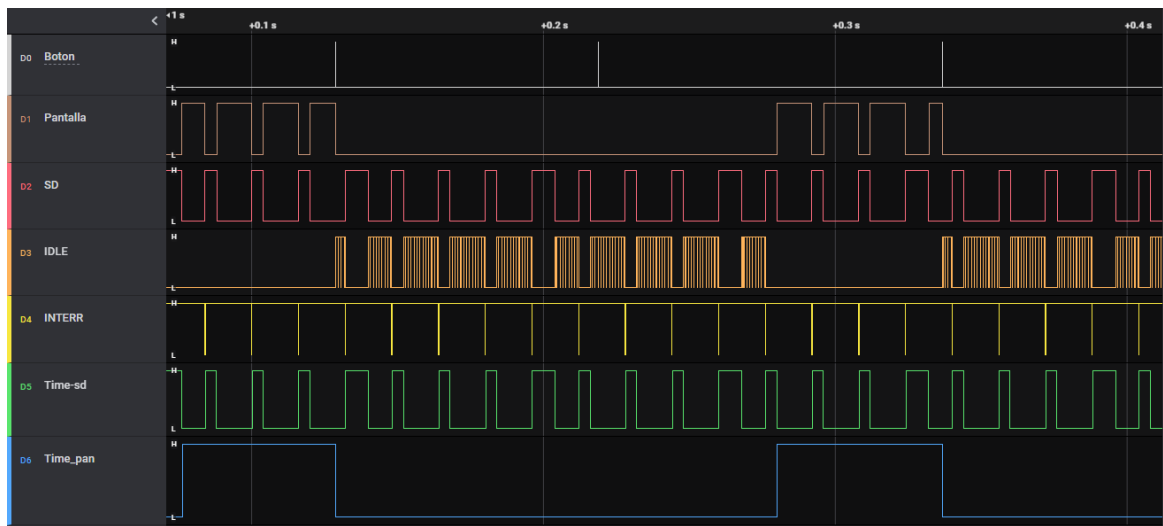


Figure 16: Logic Analyzer: con música